ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

Тема: CI/CD-пайплайн для автоматического запуска тестов
Студент: Назин Максим Сергеевич
Группа: ИСП.П 23.09.2
Дата: 28.10.26

1. Результаты выполненных заданий

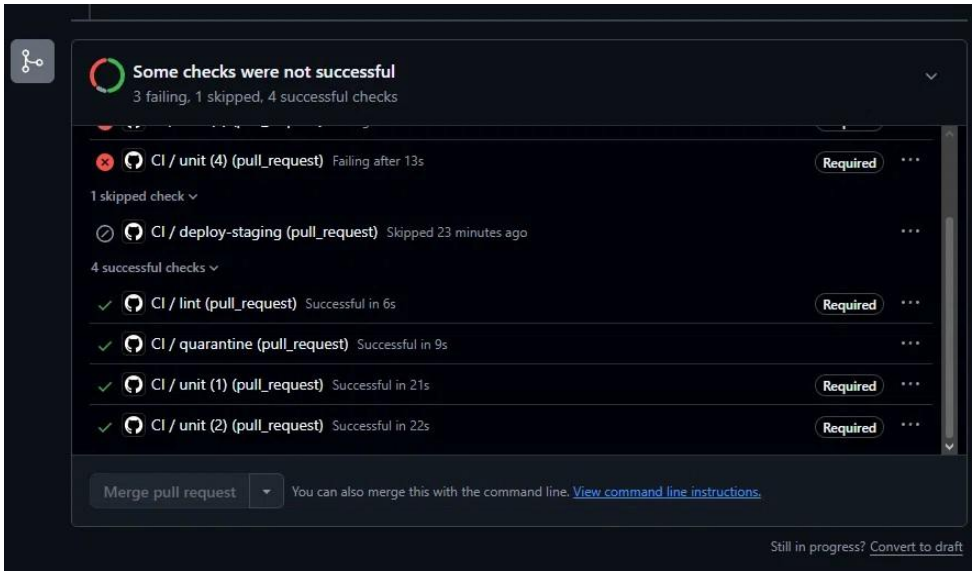
Задание 1. Подготовка репозитория

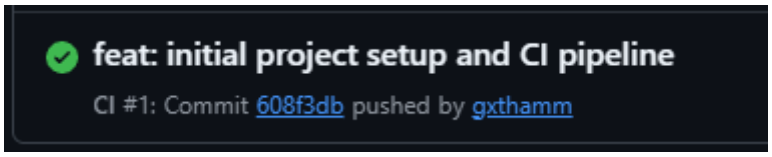
Проект инициализирован, зависимости установлены. Локальный запуск тестов (pytest -m "not quarantine" -v) и линтера (ruff check .) прошел успешно. Отсеян 1 карантинный тест, 17 тестов пройдено.

```
(.venv) PS C:\Users\gotham\Desktop\ci-practice> git push -u origin main --force
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 16 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (26/26), 6.93 KiB | 1.73 MiB/s, done.
Total 26 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/gxthamm/ci-practice.git
 + 34abc92...608f3db main -> main (forced update)
branch 'main' set up to track 'origin/main'.
(.venv) PS C:\Users\gotham\Desktop\ci-practice>
```

Задание 2. Первый workflow: lint и unit-тесты

Был создан файл ci.yml. При добавлении неиспользуемого импорта import os job lint упал, а unit был пропущен (из-за зависимости needs). После исправления кода пайплайн стал зелёным.





Задание 3. Матрица, кэш и отчёт JUnit

Настроена матрица версий Python (3.11 и 3.12) и кэширование зависимостей (cache: pip). Отчет JUnit сохраняется в Artifacts даже при падении тестов благодаря условию if: always().



Содержимое XML отчета «testsuite name="pytest" errors="0" failures="0" skipped="0" tests="17"»

Задание 4. Sharding: ускоряем прогон

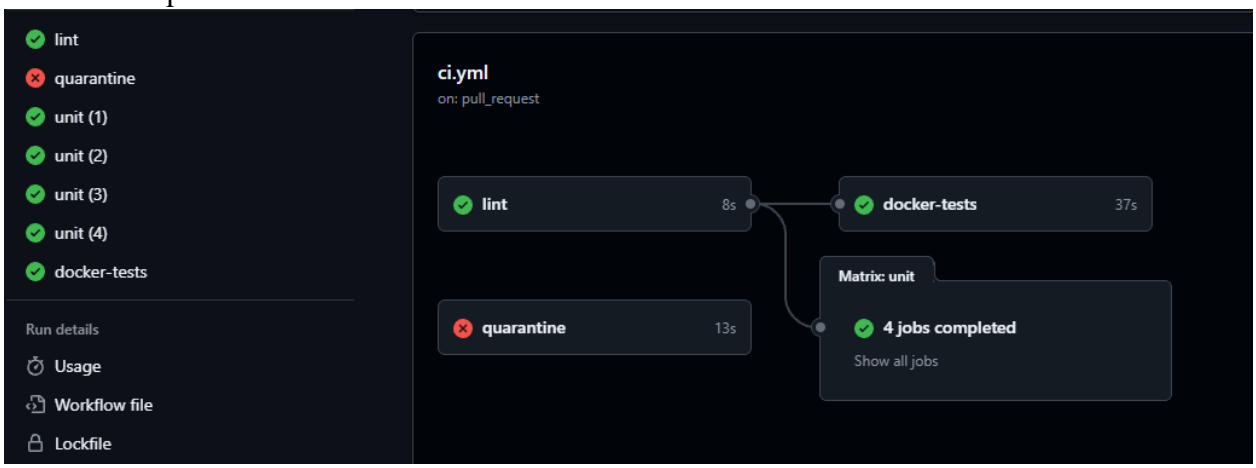
Матрица версий была заменена на 4 шарда (shard: [1, 2, 3, 4]). Параметр fail-fast: false позволяет остальным шардам завершиться, даже если один упал.

Таблица замеров:

Вариант	Число job	Время шага pytest, с	Общее время запуска, с
Без sharding	1	20	40
2 части	2	10	25
4 части	4	10	54

Задание 5. Карантин для нестабильного теста

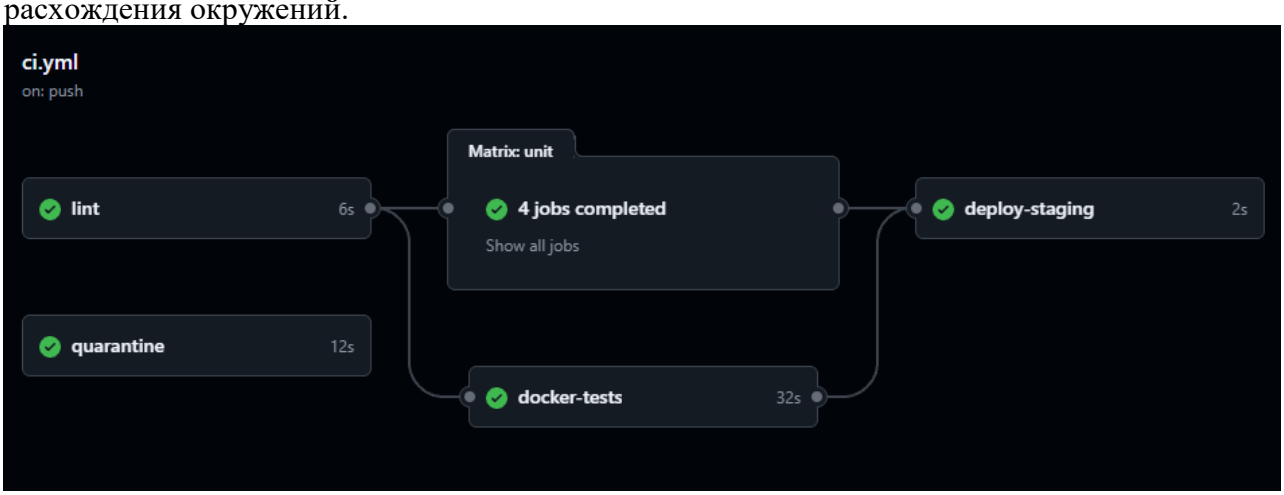
Создан test_flaky.py. При 4-5 пустых коммитах тест падал из-за своей нестабильности. Затем он был помечен маркером quarantine и исключен из основного прогона (команда pytest -m "not quarantine"). Для него добавлен отдельный job с continue-on-error: true, чтобы он не блокировал пайплайн.



https://github.com/gxthamm/gytuipy08/issues/1#issue-6079689427

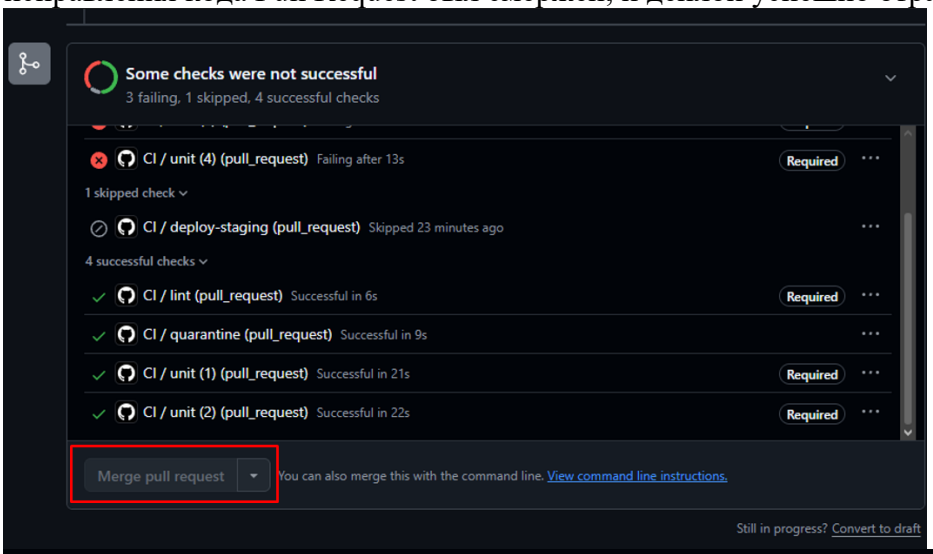
Задание 6. Тесты в Docker-контейнере

Создан Dockerfile на базе python:3.12-slim и .dockerignore. В ci.yml добавлен job docker-tests, который собирает образ и запускает тесты внутри него, решая проблему расхождения окружений.



Задание 7. Защита главной ветки и деплой-заглушка

В настройках репозитория (Settings -> Branches) для ветки main создано правило *Require status checks to pass before merging* с обязательными проверками. Добавлен этап deploy-staging, который запускается только для ветки main.
При попытке слить ветку со сломанным тестом кнопка Merge стала неактивной. После исправления кода Pull Request был смержен, и деплой успешно отработал.





2. Ответы на контрольные вопросы

1. Чем непрерывная интеграция отличается от непрерывной поставки и непрерывного развёртывания?
CI (Continuous Integration) — автоматическая сборка и тестирование кода при каждом изменении. CD (Continuous Delivery) — автоматическая подготовка артефакта к деплою (код всегда готов к выкату, но кнопку жмут вручную). CD (Continuous Deployment) — полностью автоматический деплой в продакшн без участия человека.

2. Зачем нужен параметр needs и что произойдёт с job unit, если lint упал?
Параметр needs выстраивает этапы пайплайна в воронку. Если lint упадёт, job unit получит статус Skipped (пропущен), так как запускать тесты на коде, который не проходит базовую проверку синтаксиса, бессмысленно.

3. Почему отчёт сохраняется с условием if: always()?
По умолчанию при падении тестов job останавливается. if: always() заставляет шаг с сохранением артефакта выполниться в любом случае, чтобы разработчик мог скачать JUnit XML отчет и изучить причину падения.

4. Что такое sharding и почему четыре части не дают ускорения ровно в четыре раза?
Sharding — это разбиение набора тестов на N частей (шардов) для параллельного запуска. Ускорение не в 4 раза из-за оверхеда: каждый job тратит время на запуск виртуальной машины, клонирование репозитория и установку зависимостей.

5. Чем опасен flaky-тест в основном прогоне и как работает карантин?
Flaky-тесты дают ложные падения, из-за чего команда перестает доверять пайплайну и игнорирует красный цвет. Карантин исключает их из блокирующего прогона (маркер not quarantine), но запускает в отдельном job с continue-on-error: true, а баг фиксируется в Issue.

6. Какую проблему решает Docker-контейнер при запуске тестов на CI?
Проблему «У меня работает, а на сервере нет». Контейнер фиксирует версию ОС, Python и библиотек, гарантируя идентичное окружение для тестов.

7. Почему пайплайн строят воронкой: быстрые проверки первыми?
Для экономии времени и ресурсов. Если линтер (1 минута) нашел ошибку, нет смысла запускать интеграционные тесты (15 минут). Быстрый фидбэк позволяет исправить баг сразу.

8. Какие метрики пайплайна вы бы отслеживали в реальном проекте?
Время выполнения (Duration), процент успешных/упавших сборок (Failure Rate), процент ложных падений (Flaky Rate), время от коммита до деплоя (Lead Time), покрытие кода тестами (Coverage).

3. Вывод

После внедрения CI/CD пайплайна работа с тестами стала прозрачной и автоматизированной. Теперь любое изменение кода автоматически проверяется линтером и тестами, что исключает попадание багов в главную ветку благодаря защите ветки main.

Использование шардинга (разбиения тестов) и кэширования зависимостей позволило сократить время прогона, а Docker-контейнер гарантирует одинаковое окружение на всех машинах. Появилась культура работы с нестабильными тестами через их карантин. В целом, пайплайн стал надежным фильтром, который экономит время команды на код-ревью и ручных проверках.